

Toward Multi-SLO-Aware Scheduling for Heterogeneous AI Accelerators on Edge Devices

Agenda

- ❑ Motivation and Research Challenges.

- ❑ Proposed Approach

1. Research Part #1 – Multi-SLO heterogeneous scheduling
2. Research Part #2 – Model-aware dynamic inference adaptation
3. Research Part #3 – Self-Calibrating Multi-SLO Adaptation
4. Evaluation Plan

- ❑ Timeline, Summary, and Contributions

Agenda

❑ Motivation and Research Challenges.

❑ Proposed Approach

1. Research Part #1 – Multi-SLO heterogeneous scheduling
2. Research Part #2 – Model-aware dynamic inference adaptation
3. Research Part #3 – Self-Calibrating Multi-SLO Adaptation
4. Evaluation Plan

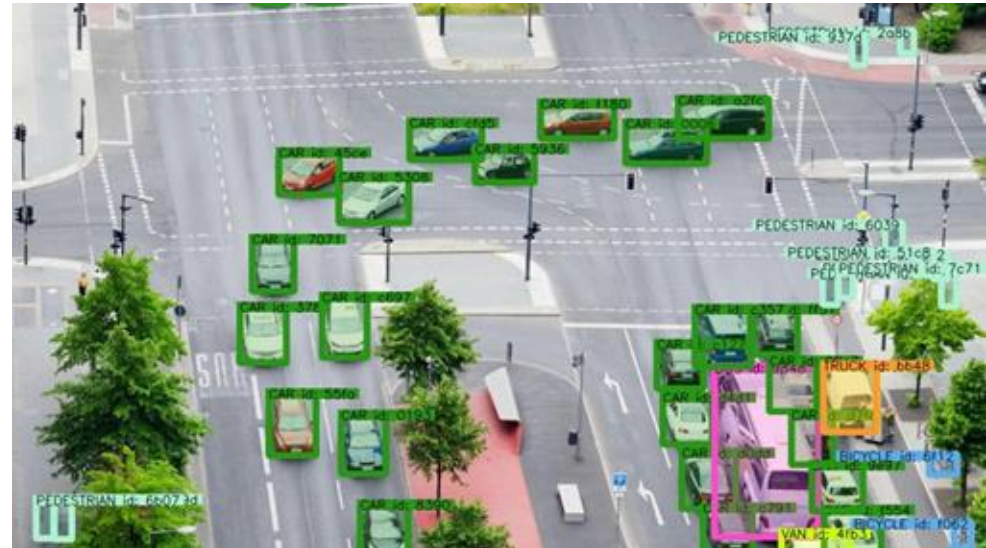
❑ Timeline, Summary, and Contributions

Research Motivation

Real-Time Smart Traffic Camera

- ❌ High latency → pedestrian detected too late → safety risk
- ❌ Accuracy drop → false negatives → missed violations
- ❌ Energy spike → thermal throttling → system slowdown

SLOs Matter



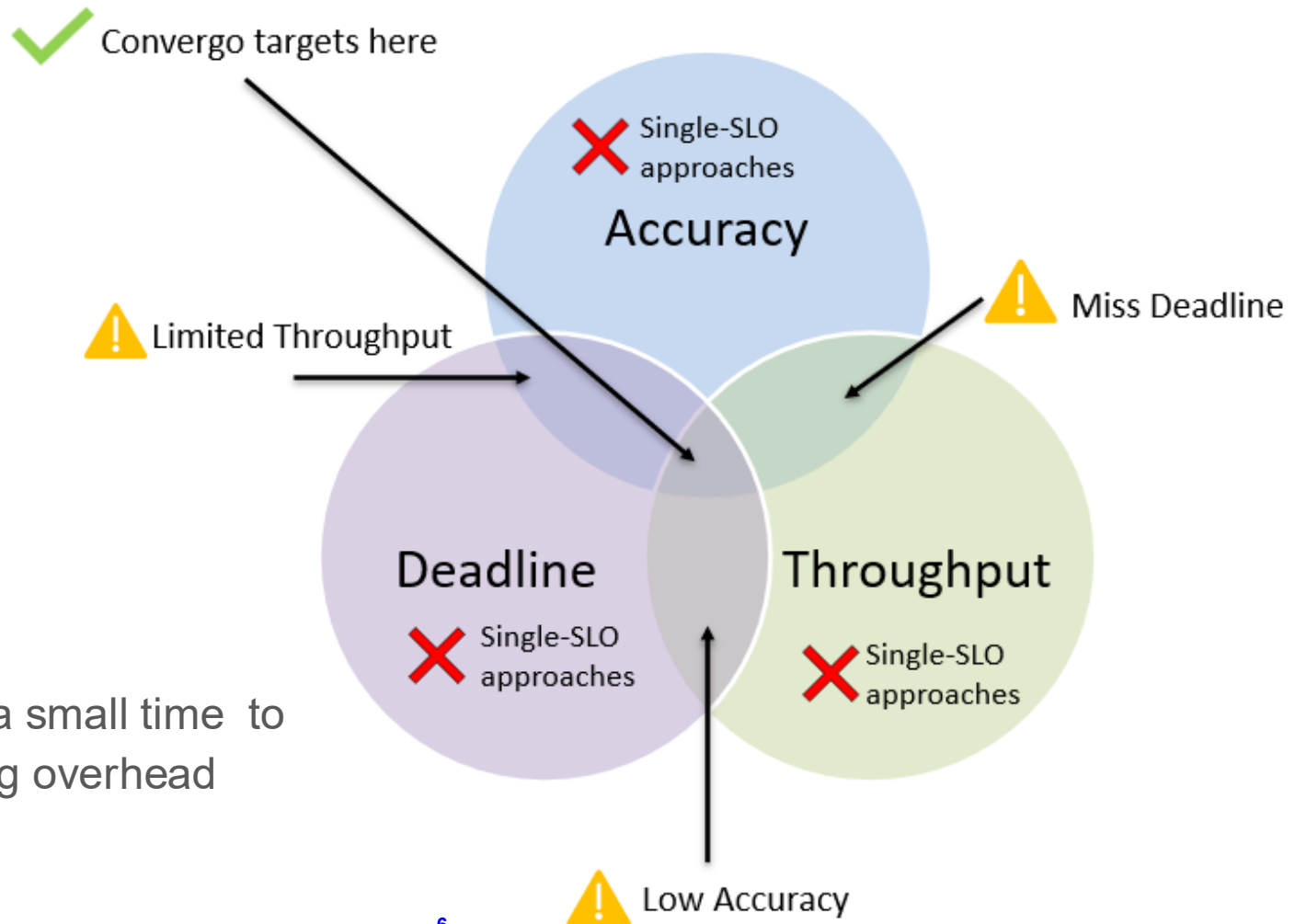
Research Challenges

Current IoT-edge Systems:

- ❑ **Multi-SLO Heterogeneous Scheduling**
- ❑ **Model-Aware Dynamic Inference Adaptation**
- ❑ **Self-Calibrating Adaptation**



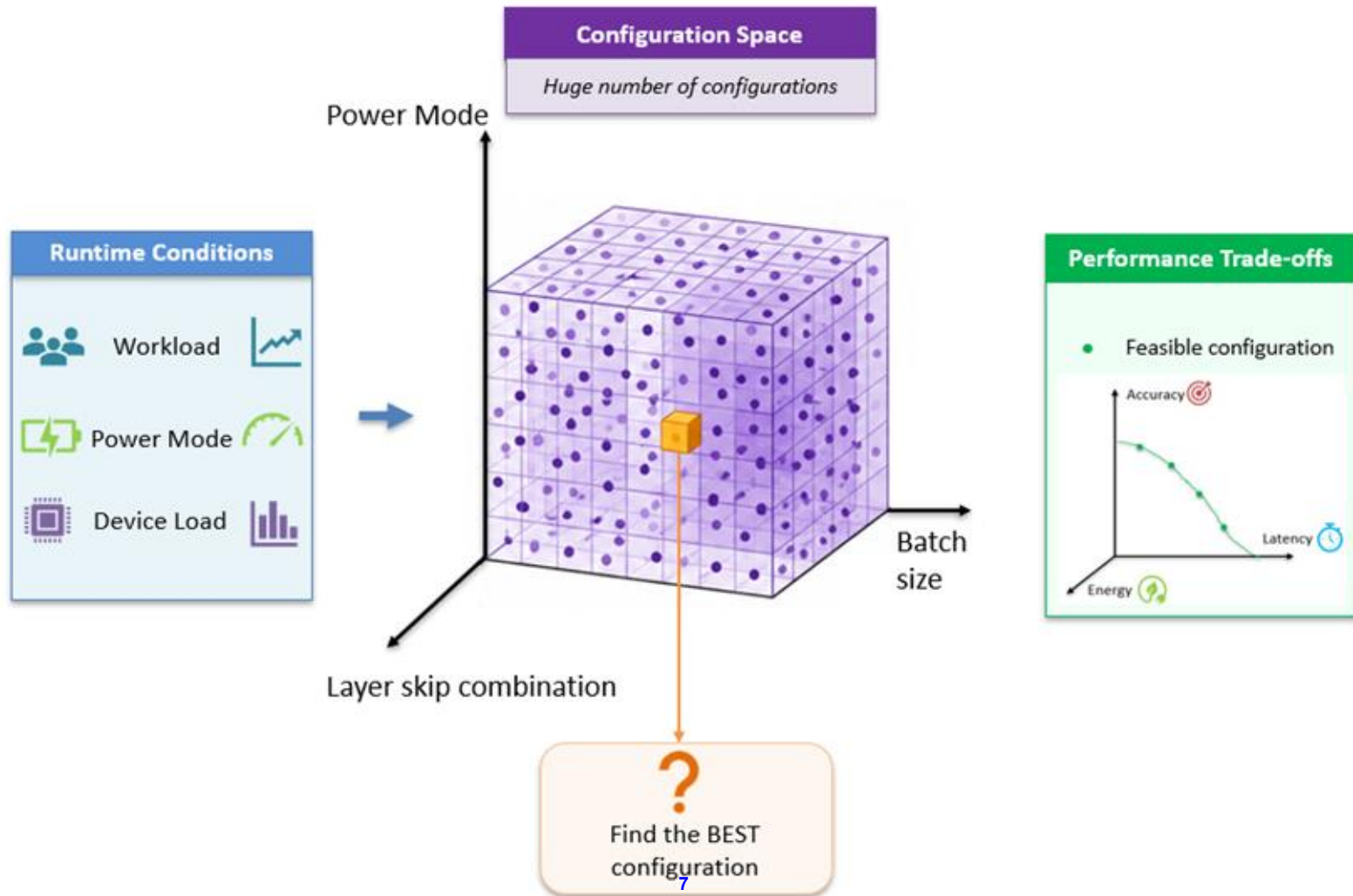
Challenge #1: Multi-SLO Heterogeneous Scheduling



- ❑ Must decide in a small time to avoid scheduling overhead

Challenge #2:

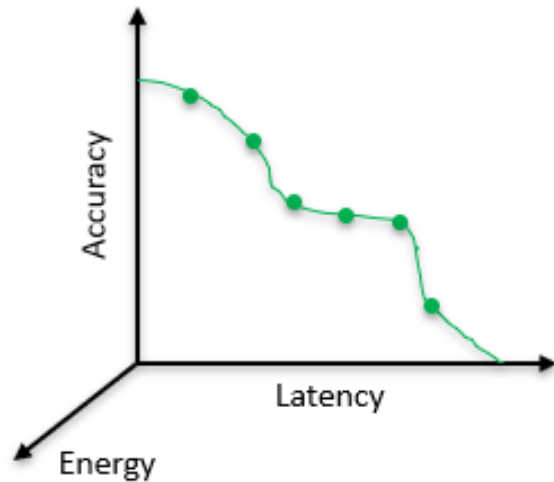
Model-Aware Dynamic Inference Adaptation



Challenge #3:

Self-Calibrating Multi-SLO Adaptation

Deployment Start



Drift
Info



Workload
Shift



Resource
Content

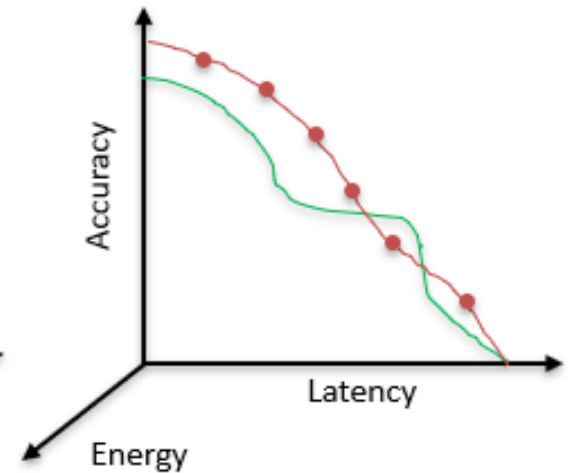


Energy
Info

Day 1 Day 30 Day 60 +



Months Later



Prediction Drift

Runtime Performance
changes over time

Problem Statement

- How can edge AI systems continuously satisfy multiple service-level objectives despite dynamic workloads, evolving device conditions, and prediction drift in heterogeneous resource-constrained environments?

Agenda

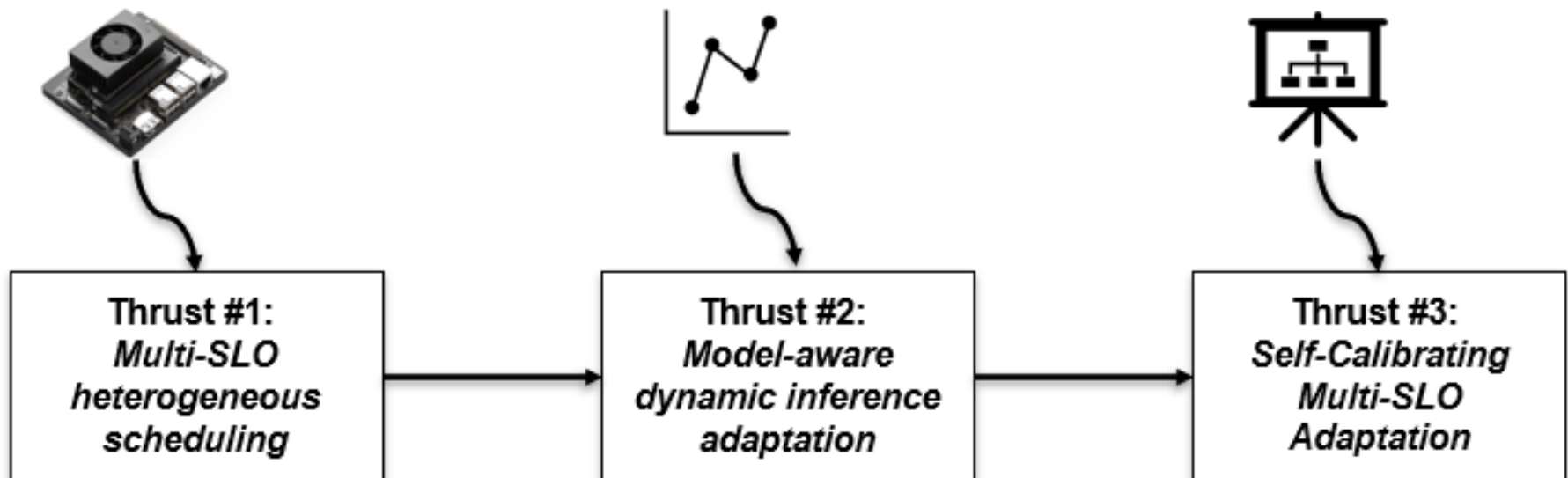
□ Motivation and Research Challenges.

□ **Proposed Approach**

1. Research Part #1 – Multi-SLO heterogeneous scheduling
2. Research Part #2 – Model-aware dynamic inference adaptation
3. Research Part #3 – Self-Calibrating Multi-SLO Adaptation
4. Evaluation Plan

□ Timeline, Summary, and Contributions

Proposed Research Overview



Research Thrust #1:

Multi-SLO heterogeneous scheduling

Research Thrust #1

❑ Challenge

- Simultaneously Meeting Multiple SLOs (Service Level Objectives) in Edge AI Scheduling

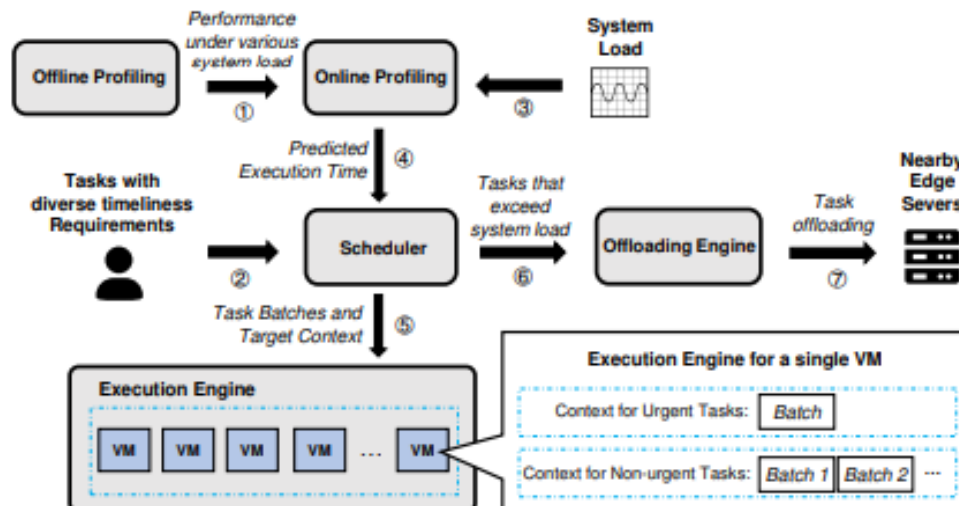
❑ Research goal

- Establish the foundational system-level orchestration layer that intelligently coordinates heterogeneous accelerators and static models to maximize multi-SLO satisfaction, while revealing the fundamental efficiency ceiling imposed by the black-box model paradigm.

Research Thrust #1 -- Related Work

□ Park, Seonyeong, et al. (INFOCOM'24)

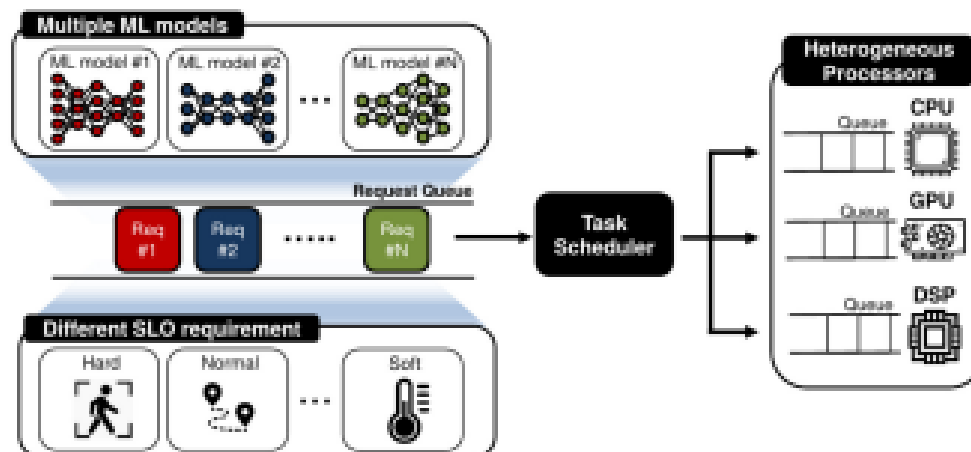
- Kalmia is a scheduling framework for edge servers that manages heterogeneous DNN inference tasks while meeting QoS requirements by co-locating different types of workloads (latency-critical and best-effort) on GPUs and CPUs.



Research Thrust #1 -- Related Work

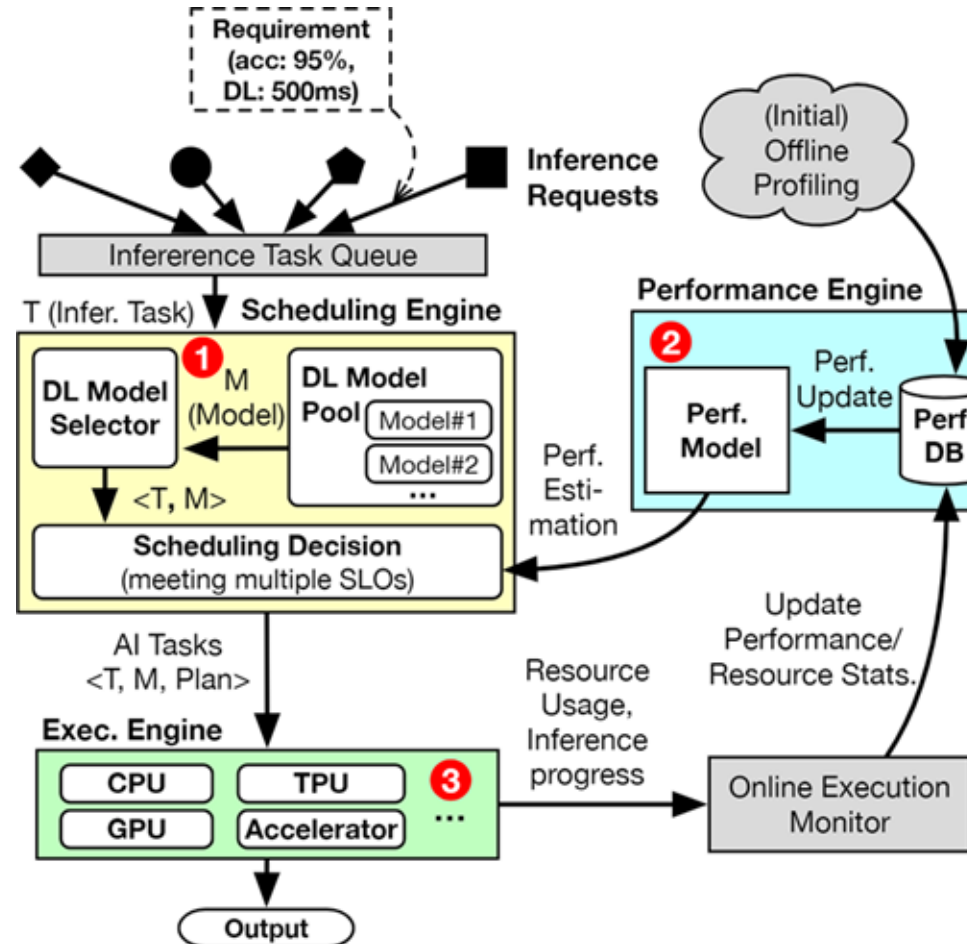
❑ Seo, Wonik, et al. (ACM TACO'21)

- This paper presents an SLO-aware scheduler for DNN inference on heterogeneous edge platforms (CPUs, GPUs, and accelerators) that dynamically assigns inference tasks to different processors to meet service level objectives (SLOs). The scheduler uses runtime profiling and prediction models to estimate execution times and makes scheduling decisions that maximize resource utilization while satisfying latency requirements.



Research Thrust #1

- 1 Scheduling Engine**
 - Select best models to meet SLOs
- 2 Performance Engine**
 - Profiles and estimate model perf.
- 3 Execution Engine**
 - Runs models on AI accelerators



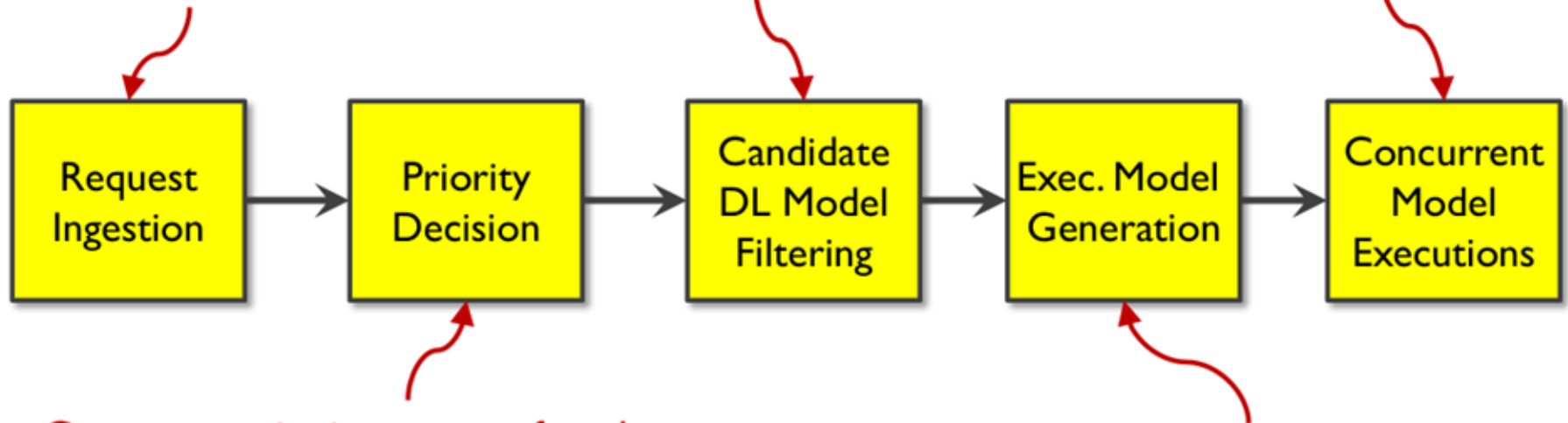
Research Thrust #1

□ Scheduling Overview

New inference requests arrive at runtime

Finding proper DL models that are likely to meet all SLOs

Run models in batches



Compute priority score of each request based on deadline and weight

Build batch-wise model placements on GPUs/TPUs

Research Thrust #1

❑ User Requests and Priority

- Convergo handles multiple concurrent inference tasks with dynamic priority scheduling.

❑ Each user request (R_i) includes multiple SLOs:

- Throughput (Th_i), Accuracy (Acc_i), Deadline (Di)

❑ All requests are placed in a **priority queue**

$$Priority_i = \frac{D_i - (t_{curr} - t_{arr, R_i})}{w_i},$$

The diagram includes the following annotations:

- Deadline**: An arrow points from this label to D_i in the numerator.
- Current Time**: An arrow points from this label to t_{curr} in the numerator.
- Arrival Time**: An arrow points from this label to t_{arr, R_i} in the numerator.
- User assigned weight (1 -- 10)**: An arrow points from this label to w_i in the denominator.

❑ Priority scores are dynamically updates over time

Research Thrust #1

- ❑ After prioritization, Convergo checks 3 constraints to find candidate DL models.

- Constraint #1: Throughput

$$\sum_{j=1}^n \mathcal{M}_j(Th) \times t_j \geq Th_i,$$

- Constraint #2: Accuracy

$$\frac{\sum_{j=1}^n \mathcal{M}_j(Th) \times t_j \times \mathcal{M}_j(Acc)}{\sum_{j=1}^n \mathcal{M}_j(Th)} \geq Acc_i,$$

- Constraint #3: Deadline

$$\sum_{j=1}^n t_j \leq D_i.$$

- ❑ Among the candidates, choose those with minimal total execution time (to maximize slack).
- ❑ Generate exec. plan from the selected models

Research Thrust #1

❑ Edge Platform: Nvidia Jetson Xavier

- 6-core CPUs, 8G RAM, Volta GPUs

❑ Augmented with multiple TPU acc.

❑ AI Applications and Models

- 4TOPS @ 2W



Jetson Xavier



Coral
TPU Accelerator

3 Diff
Apps

AI Application	Edge Accelerator	Pre-trained DL Models	Size (MB)	Num. Layers
Image Classification	GPU	EfficientNet [36]	5.4	5
		Inception-v1 [34]	6.4	22
		MobileNet-v2 [29]	3.4	20
		SqueezeNet [16]	1.3	15
	TPU	EfficientNet	6.8	7
		Inception-v1	7.0	22
		Inception-v3 [35]	12.0	48
		MobileNet-v1 [15]	1.6	28
Object Detection	GPU	EfficientNet	4.5	5
		MobileNet-v2	6.5	20
		YOLOX [12]	5.4	11
	TPU	EfficientNet	7.6	18
		MobileNet-v1	7.0	28
		MobileNet-v2	6.6	20
		MobileNet-v1 [15]	8.2	28
Pose Estimation	GPU	MoveNet [3]	5.6	25
		PoseNet [25]	4.3	23
		MoveNet	7.5	25
	TPU	PoseNet-MobileNet [25]	1.5	23
		PoseNet-ResNet [25]	24.4	18

GPU Models
on TensorFlow

TPU Models
on TFLite

Research Thrust #1

❑ Workload Generation

- Mixed AI tasks: IC, OD, PE
- 500 reqs per workload, generated using Poisson distribution
- Each req includes unique SLOs: accuracy, throughput, and deadline
- Evaluated over 10 workload sets to reduce bias and ensure robustness

❑ Baselines

- SLO-MAEL (ACM TACO'21): Prioritizes based on estimated latency and risk of SLO violation; uses preemptive execution
- Kalmia (INFOCOM'24): QoS-aware scheduler for heterogeneous DNNs; combines preemptive and context-aware policies

Research Thrust #1 – Preliminary Results

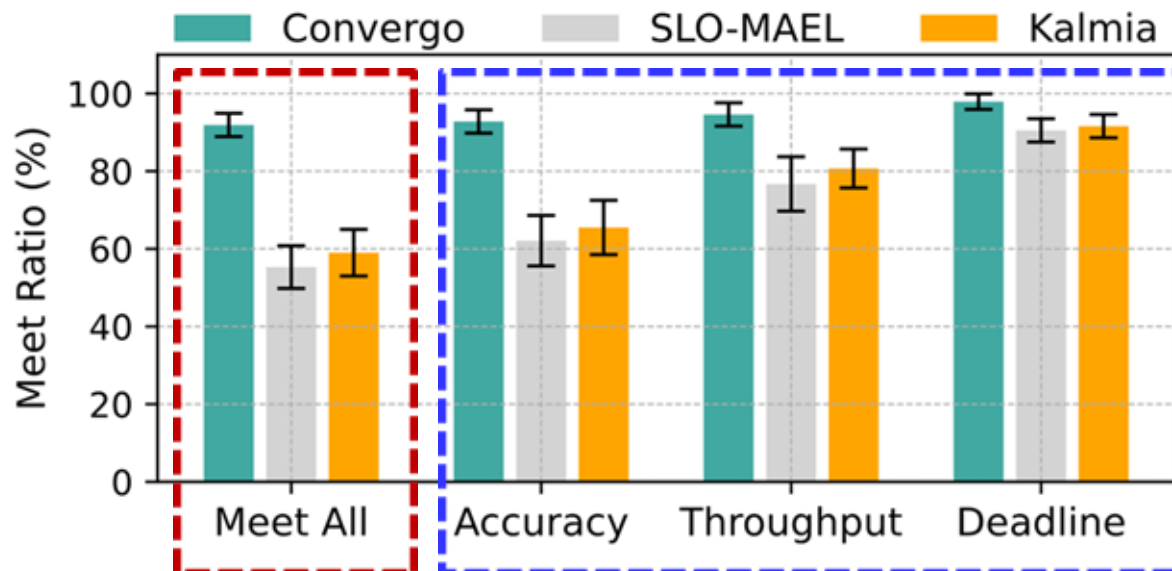
- Measured both “Meet All” and individual SLOs
 - Meet All: satisfying all three SLOs simultaneously

Meet All:

- Convergo: over 95%
- Baselines: less than 60%

Individual SLOs:

- Convergo: 93% (acc), 95% (th), 98% (DL)
- Baselines: less than 70% (acc), less than 80% (th)

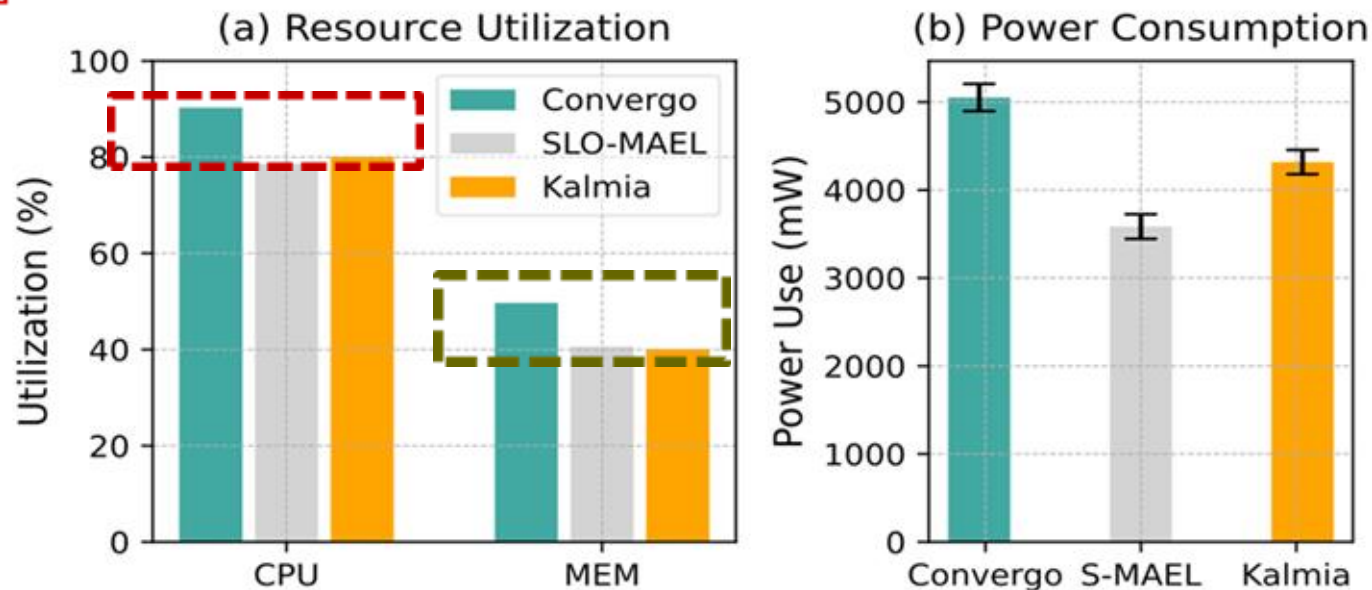


Research Thrust #1 – Preliminary Results

Resource Utilization

CPU util
increased
by 12%

MEM util increased by 10%



Research Thrust #1 – Preliminary Results

❑ Covergo Scheduler Overhead

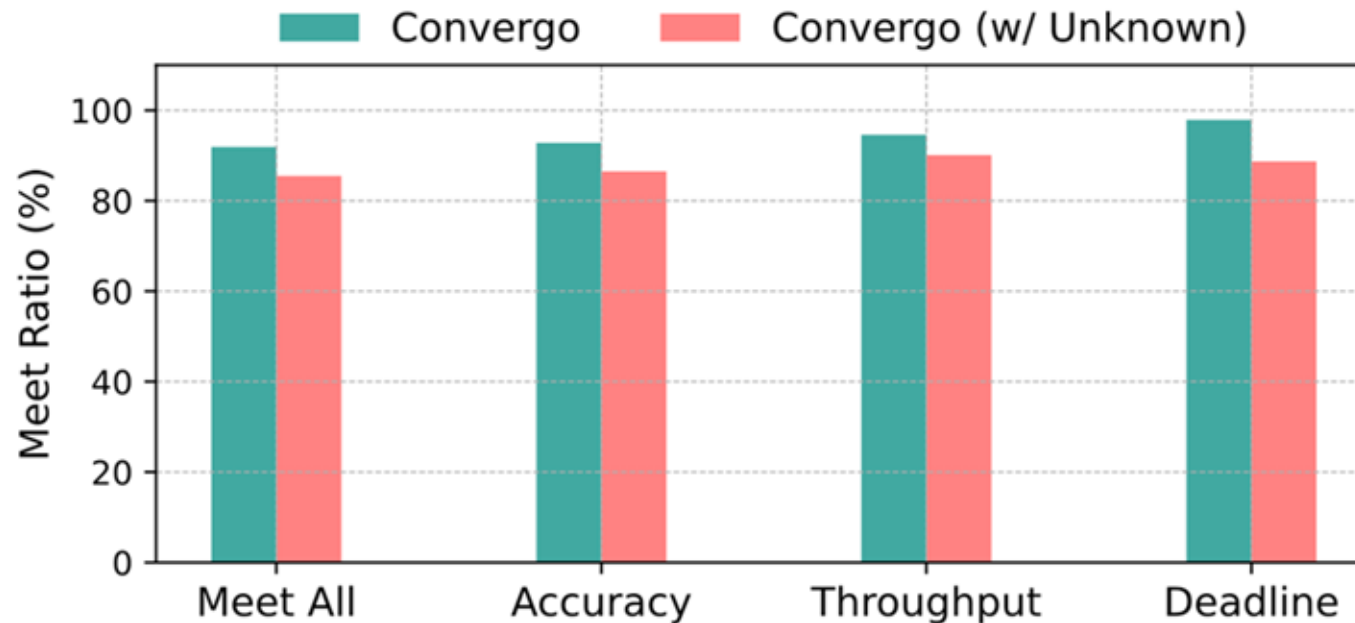
	Avg	Std Dev
Extra CPU <u>Consume..</u>	6.3%	1.6
Extra MEM Consume.	4.9%	1.1
Extra Power Consume.	527mW	45

Still, less than 10% of dev power.

Research Thrust #1 – Preliminary Results

Performance with ‘unknown’ Models

- 30% of models were previously unknown (i.e., unprofiled).



Research Thrust #2:

Model-aware dynamic inference adaptation

Research Thrust #2

□ Research goal

Develop skip-aware models and lightweight prediction mechanisms that expose a quantifiable accuracy performance trade-off space for safe and efficient runtime adaptation.

Research Thrust #2

❏ Challenge

Real-time Multi-Objective Adaptation under Dynamic Runtime Conditions

- Configuration effects are highly coupled:
- Runtime conditions are non-stationary:
- Decisions must be made online:
- Existing optimizers (e.g., BO) are too slow for per-inference adaptation

Research Thrust #2

❑ From BO to Learning-Based Runtime Prediction

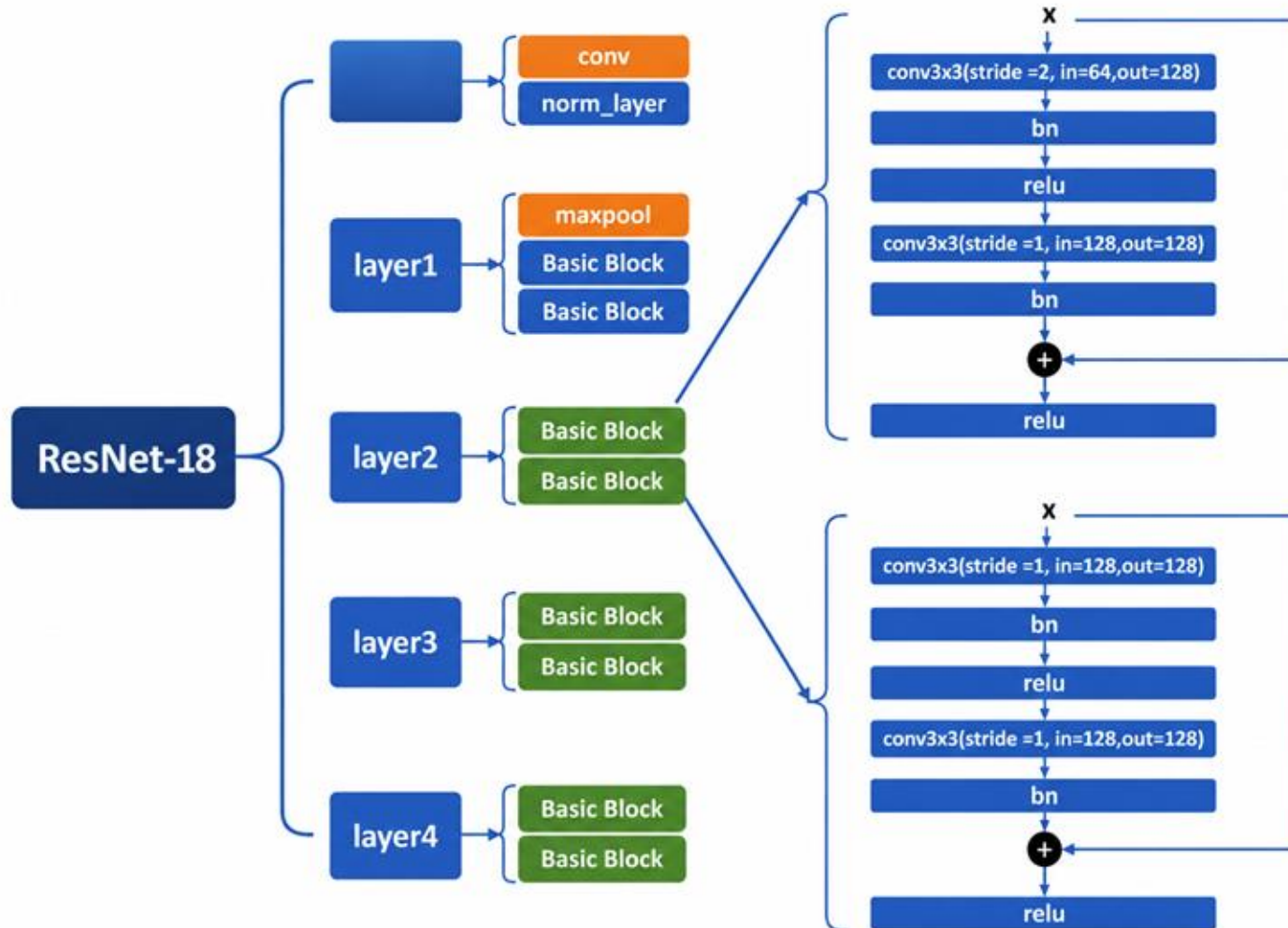
❑ Bayesian Optimization (BO) Is Not Enough:

- expensive but infrequent evaluations
- stationary objective function
- slow convergence acceptable

❑ Our Approach:

- trains lightweight performance predictors offline
- predicts latency/energy/accuracy in milliseconds
- searches the configuration space using predictor outputs
- enable millisecond multi-SLO decisions under changing runtime conditions

Research Thrust #2



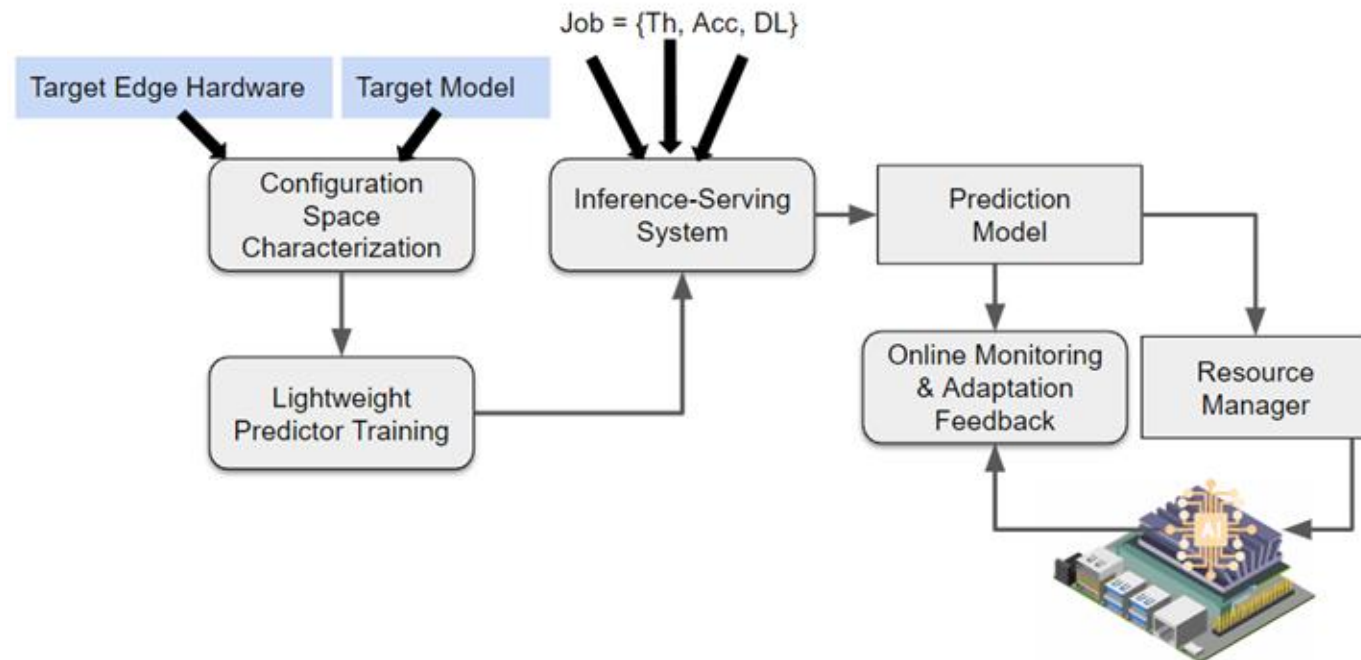
Research Thrust #2

Offline model

Enumerates a set of allowable skip configurations

Online stage

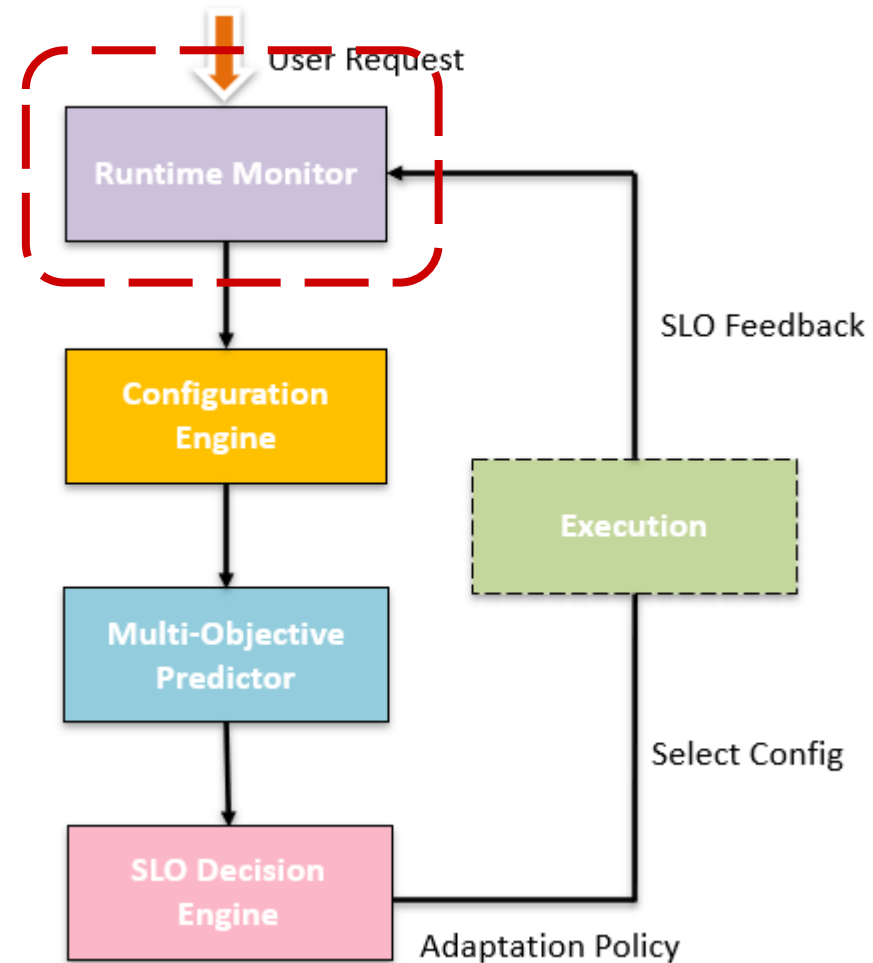
Select a suitable skip configuration



Research Thrust #2

□ Runtime Monitor

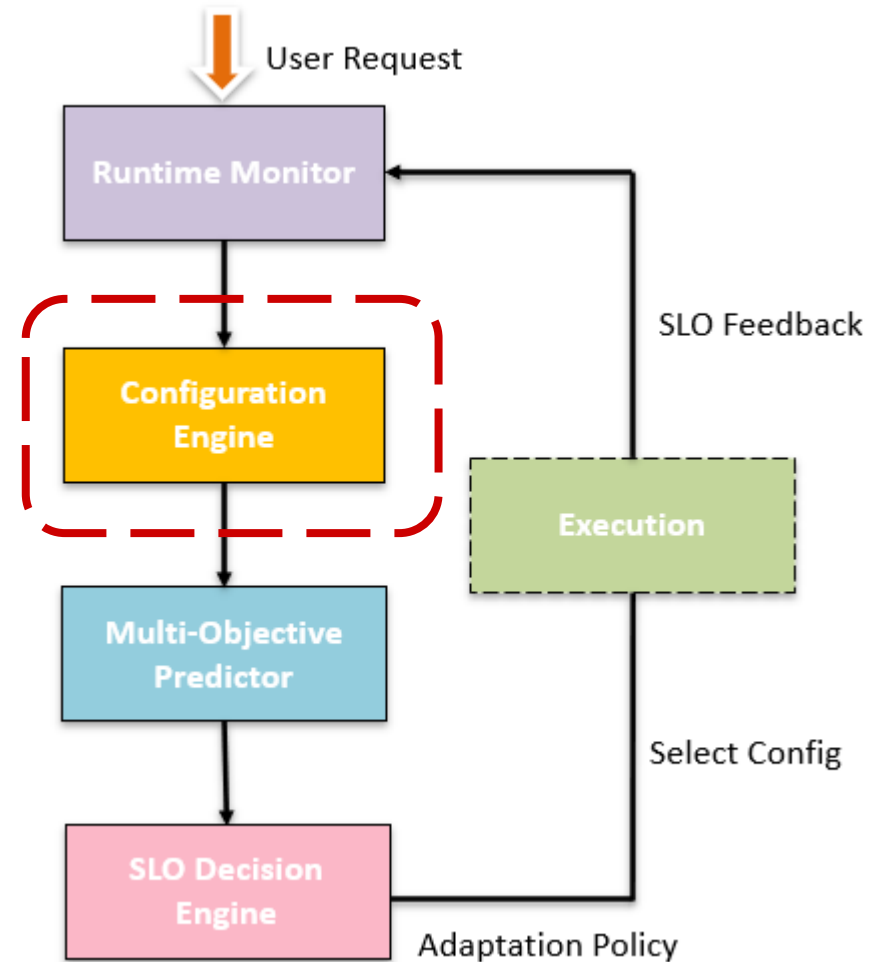
- Power State
- Device Load
- Queue Length



Research Thrust #2

□ Configuration Engine

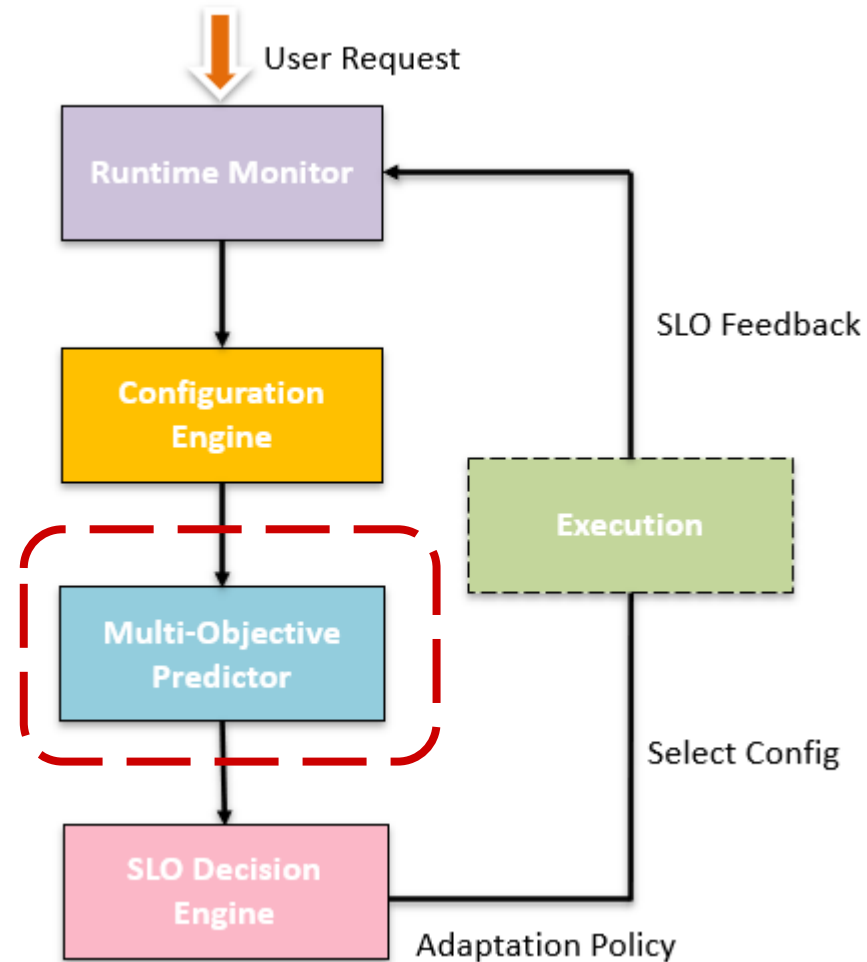
- Skip Config
- Batch Size
- Power Mode



Research Thrust #2

❑ Multi-Objective Predictor

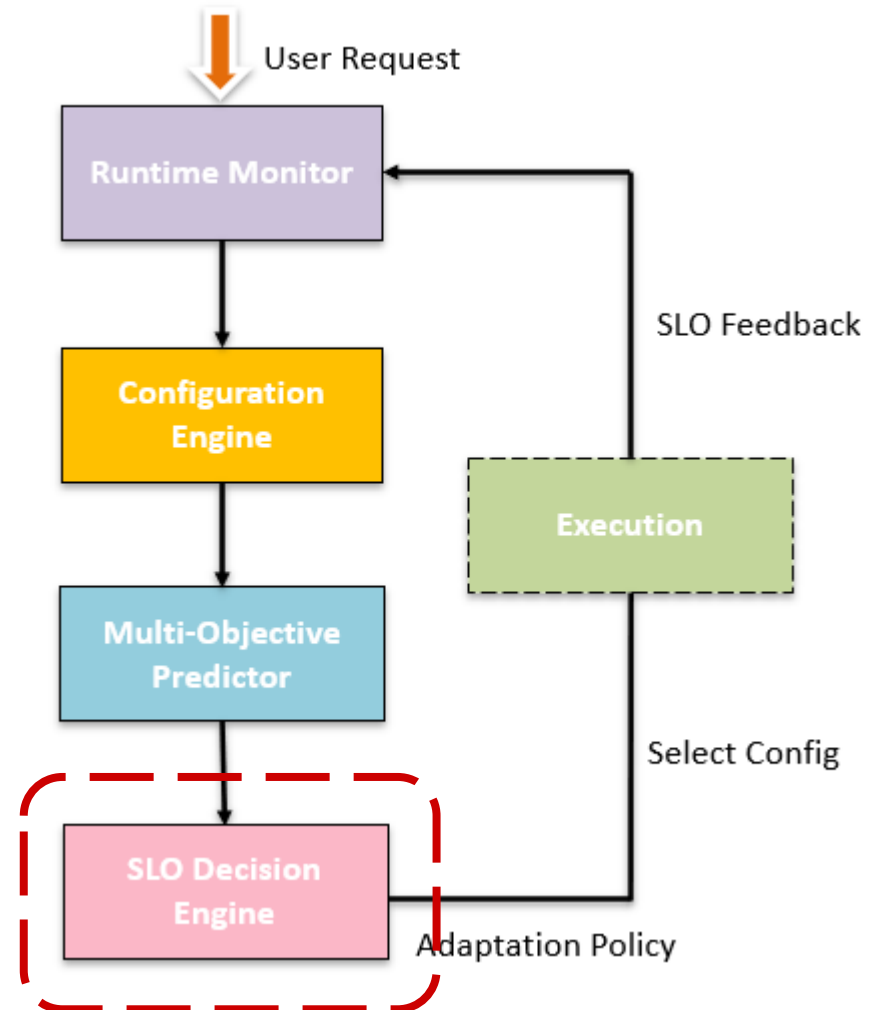
- Accuracy Predictor
- Latency Predictor
- Energy Predictor



Research Thrust #2

❑ SLO Decision Engine

- Accuracy Constraint
- Deadline Constraint
- Energy Objective

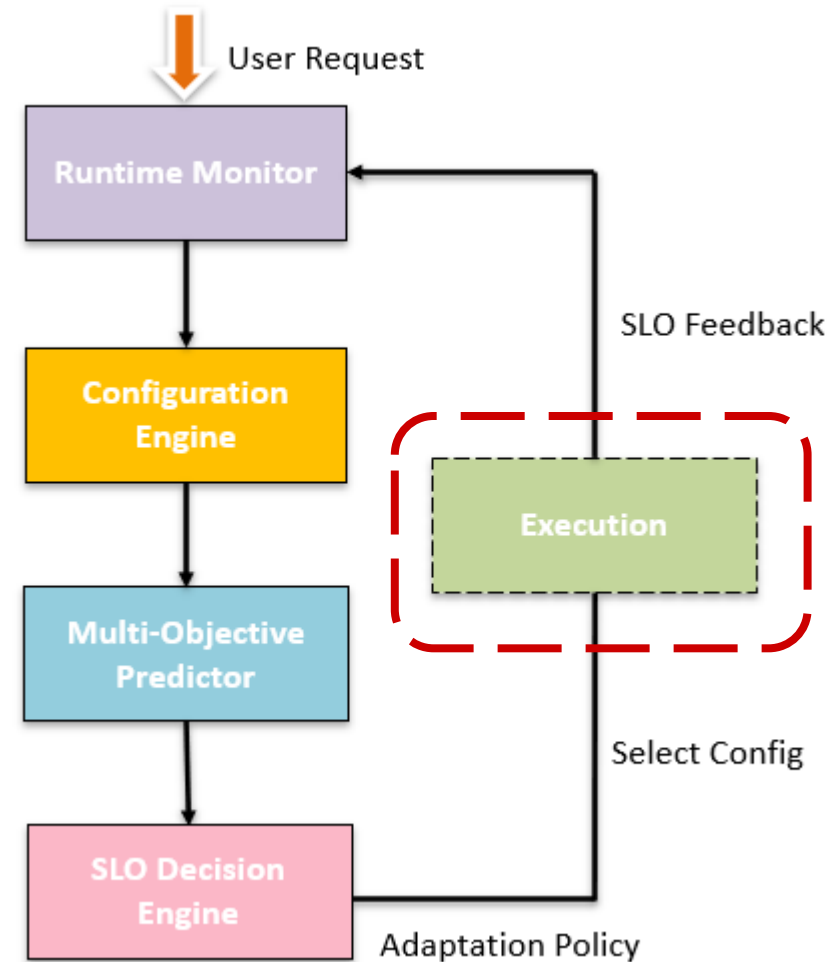


Research Thrust #2

❑ Execution

Execution on heterogeneous devices

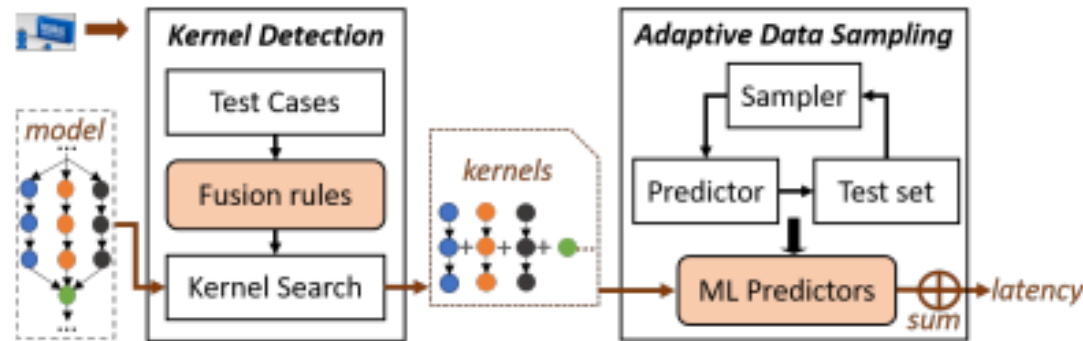
- Jetson Orin Nano
- Jetson AGX Xavier
- GPU



Research Thrust #2 -- Related Work

□ Zhang, Li Lyna, et al. (MobiSys'21)

- Proposes a learning-based framework to predict DNN inference latency on diverse edge devices.
- Builds a latency predictor using operator-level profiling and regression modeling.



□ Limitations

- Predicts latency only; does not model accuracy, energy consumption, or multi-objective trade-offs needed for SLO satisfaction.
- Requires extensive offline kernel profiling for each new device; adaptation overhead limits real-time reconfiguration scenarios.
- Static prediction model cannot adapt to dynamic system conditions

Research Thrust #2 -- Related Work

❏ Wang et al. (ECCV 2018)

- Introduces dynamic layer-skipping through learnable gating networks.
- Routes each input through different residual blocks based on input difficulty.
- Reduces inference execution.



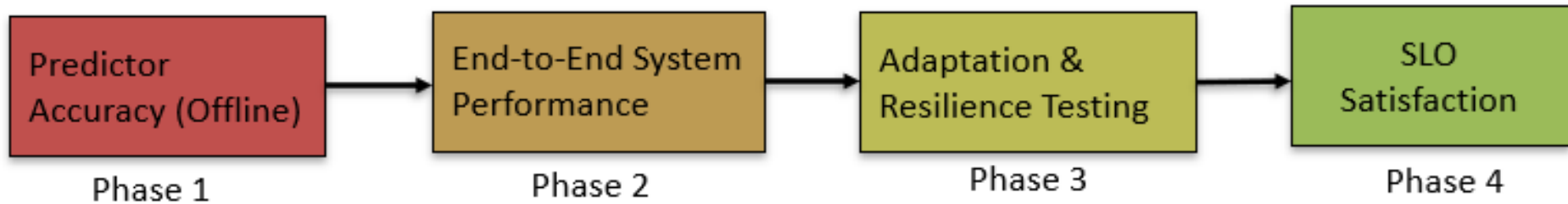
❏ Limitations

- Optimizes inference efficiency only; does not consider energy-aware multi-SLO objectives.
- Decisions depend primarily on input features, not runtime system conditions.
- Assumes a fixed deployment environment without workload-aware adaptation.
- Does not explicitly model accuracy–latency–energy tradeoff surfaces for runtime optimization.

Evaluation Plan

- ❑ RQ1: How accurate are our lightweight prediction models (latency + accuracy)?
- ❑ RQ2: Can dynamic layer-skipping meet diverse SLOs while reducing energy?
- ❑ RQ3: What is the overhead of runtime decision-making?
- ❑ RQ4: How does the system adapt to workload variations?

Evaluation Plan



Evaluation Plan

Hardware Platforms:

Device	CPU	GPU	Memory	Power	Price	Purpose
Jetson Orin Nano	6-core Arm Cortex-A78AE	1024-core NVIDIA Ampere	8GB	7–15W	\$499	Low-power edge deployment
Jetson AGX Xavier	8-core Carmel ARM	512-core Volta + Tensor	32GB	10–30W	\$699	High-end edge server
GeForce RTX 2080 Ti	8-core 64-bit CPU	4352 CUDA cores	16GB	250W	\$999	Desktop GPU baseline
RTX A5000	48-core 64-bit CPU	8192 CUDA cores	196GB	230W	\$2500	workstation GPU

Evaluation Plan

- ❑ Models:

ResNet-18, ResNet-34, ResNet-50, ResNet-74, ResNet-101,
ResNet-152

- ❑ Dataset: CIFAR-10 and CIFAR-100

- ❑ Layer-skip configurations: 4,095 paths per model (ResNet-34 example)

- ❑ Batch sizes: {1, 2, 4, 8, 16, 20}

- ❑ Power modes: 8 combinations (4 modes $\times$ jetson_clocks on/off)

- ❑ Total: About 160,000 unique configurations per model

Evaluation Plan

Baselines:

❑ B1: nn-Meter (MobiSys'21)

- Builds device-specific performance models using operator-level profiling
- Predicts latency only

❑ B2: Dělen (IoTDL'23)

- Dynamically adjusts batch size and power modes
- Limited multi-objective optimization

❑ B3: MSDNet (ICLR'18)

- Multi-exit neural architecture with adaptive early termination
- Not designed for multi-SLO scheduling

❑ B4: SkipNet (ECCV'18)

- Dynamically skips residual blocks using learned gating networks
- Optimizes model execution only

Evaluation Plan

Comparison Matrix:

System	Runtime Adaptive	Accuracy-Aware	Energy-Aware	Multi-SLO
nn-Meter	X	X	X	X
Dělen	✓	Partial	Partial	Partial
MSDNet	Partial	✓	✓	X
SkipNet	✓	✓	X	X
Thrust 2	✓	✓	✓	✓

Evaluation Plan

Expected Results:

- ❑ Prediction Accuracy
- ❑ SLO Satisfaction
- ❑ Energy Savings
- ❑ Overhead
 - Prediction time
 - Total overhead

Research Thrust #3:

Self-Calibrating Multi-SLO Adaptation

Research Thrust #3

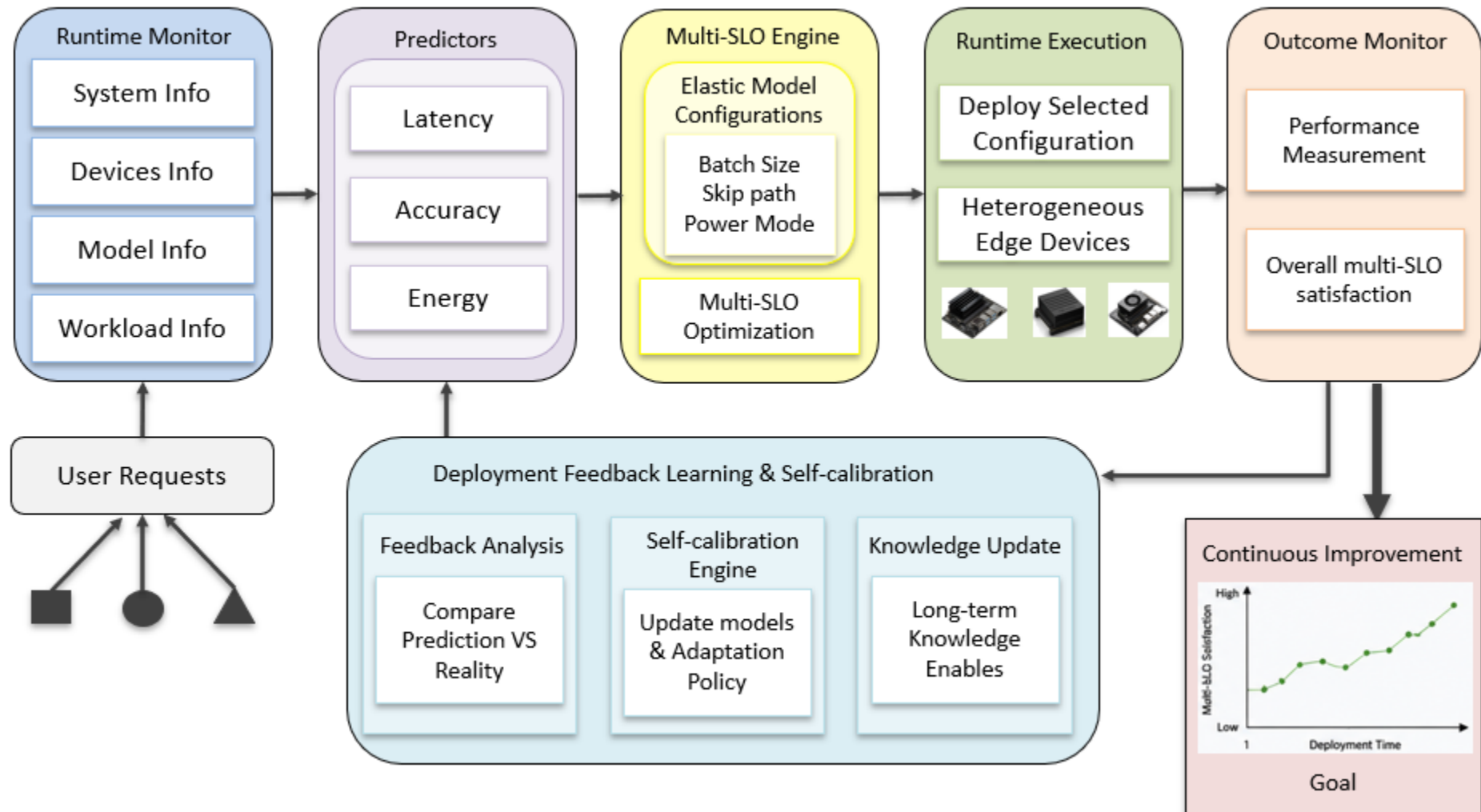
❏ Challenge

- Runtime environments are non-stationary
- Static adaptation policies cannot correct deployment drift
- Prediction models gradually become stale

❏ Research goal

- Develop deployment-feedback mechanisms that continuously calibrate adaptation models under evolving workload and device conditions.

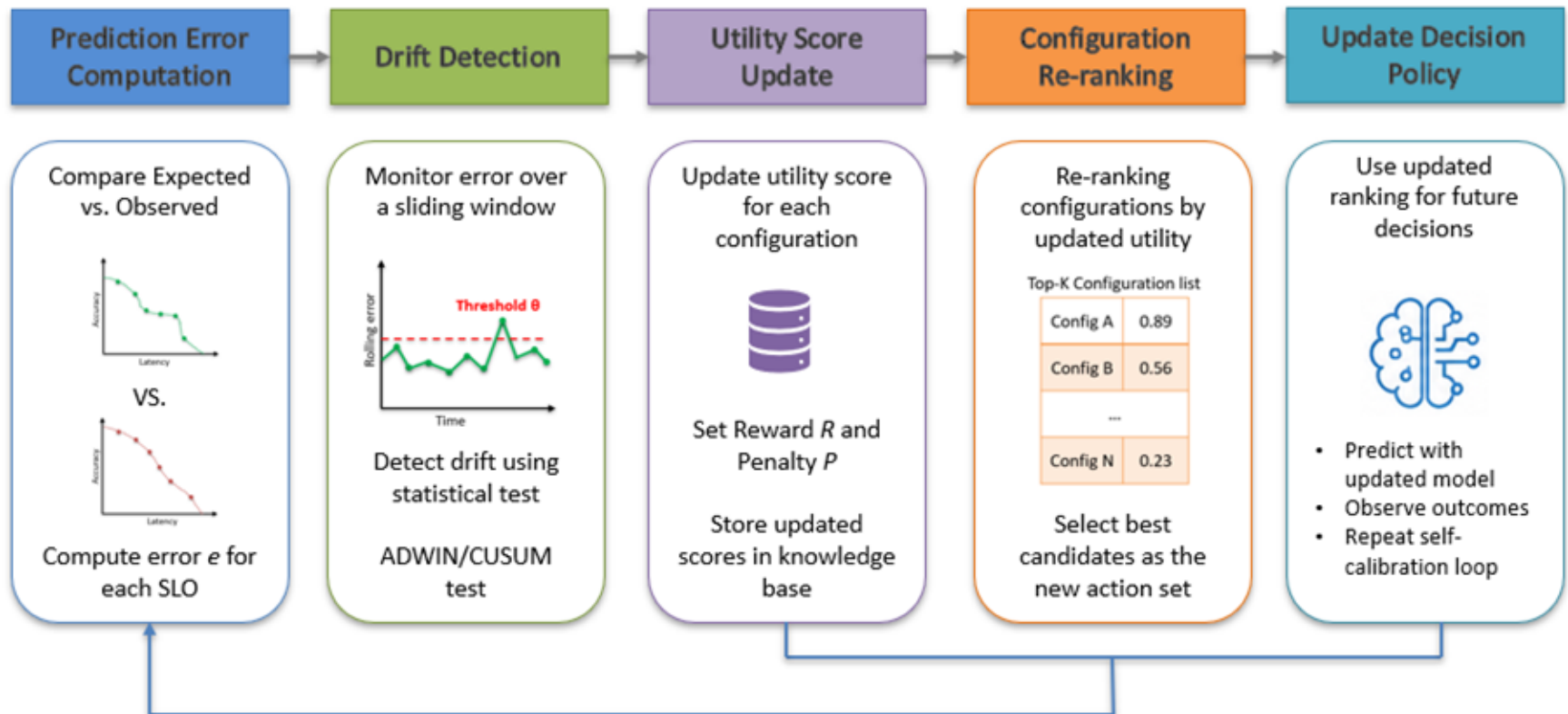
Research Thrust #3



Research Thrust #3

Deployment Feedback Learning & Self-calibration Goal:

Develop a lightweight drift-aware self-calibration framework that maintains multi-SLO adaptation under deployment drift without expensive online model retraining.



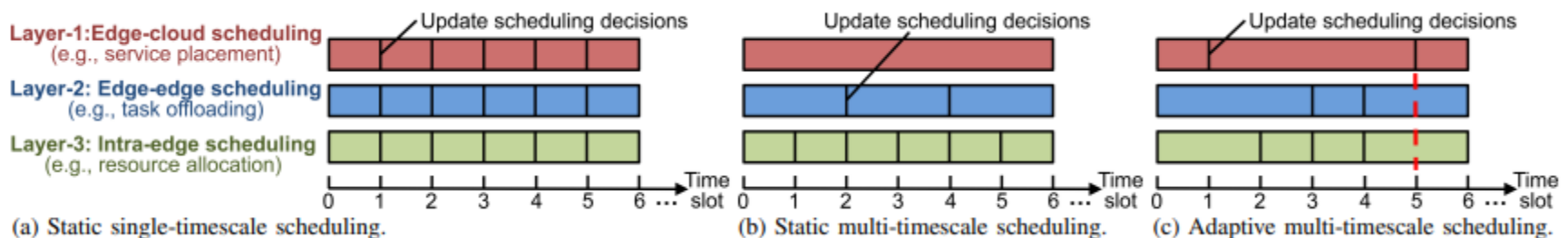
Research Thrust #3 -- Related Work

□ Hao, Y. et al. (ICASSP' 24)

- Proposes a deep reinforcement learning framework for adaptive scheduling in mobile edge computing environments.
- Learns scheduling policies across multiple time scales to handle dynamic workloads and resource variations.
- Continuously adapts scheduling decisions based on observed system feedback.

□ Limitations

- Assumes fixed application models and does not consider internal model configurations.
- Does not optimize accuracy, latency, and energy jointly.



Research Thrust #3 -- Related Work

❑ Wirth, M. et al. (GLOBECOM '23)

- Uses contextual multi-armed bandits to make runtime decisions in non-stationary edge environments.
- Learns from historical observations and continuously updates decision policies.

❑ Limitations

- Focuses on server selection and task offloading decisions.
- Does not consider elastic DNN configuration spaces.
- Does not address multi-SLO optimization.

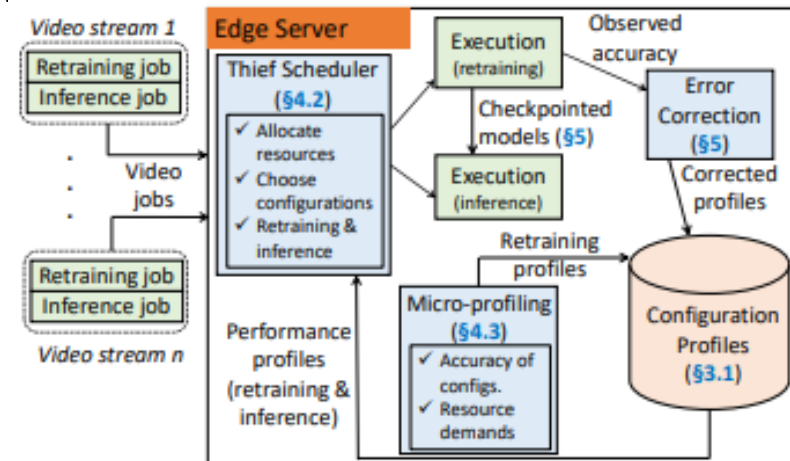
Algorithm 1 C-CPD-UCB

```
1: Input Parameters:  $T, W, \xi$  and  $\delta$ 
2: Set  $N_{k,0}(\alpha) = 0, \forall k, \forall \alpha$ .
3: Set  $\tau = 0$ .
4: for each  $t = 1, \dots, T$  do
5:   Observe task type  $\alpha_t$ .
6:   if  $N_{k,t-1}(\alpha_t) > 0 \forall k \in \mathcal{K}$  then
7:     Select BS  $\hat{k}_t$  according to (10).
8:   else
9:     Select BS  $\hat{k}_t$  randomly from set  $\{k \in \mathcal{K} | N_{k,t-1}(\alpha_t) = 0\}$ .
10:  end if
11:  Observe delay  $T_{\hat{k}_t,t}(\alpha_t)$  and update  $\hat{\mu}_{\hat{k}_t,t}(\alpha_t)$ ,  $N_{\hat{k}_t,t}(\alpha_t)$  and  $c_{\hat{k}_t,t}(\alpha_t)$ 
    based on Equations (11), (13) and (12), respectively.
12:  if Equation (14) is satisfied AND  $N_{k,t}(\alpha) > 0 \forall k \in \mathcal{K}, \forall \alpha \in \mathcal{A}$  then
13:    Set  $\tau = t - W$ .
14:    for  $k \in \mathcal{K}$  do
15:      for  $\alpha \in \mathcal{A}$  do
16:        Update  $N_{k,t}(\alpha)$  according to Equation (13).
17:        if  $N_{k,t}(\alpha) > 0$  then
18:          Update  $\hat{\mu}_{k,t}(\alpha)$  and  $c_{k,t}(\alpha)$  based on Equations (11) and (12),
            respectively.
19:        end if
20:      end for
21:    end for
22:  end if
23: end for
```

Research Thrust #3 -- Related Work

□ Bhardwaj, R., et al. (USENIX NSDI'22)

- Introduces a continuous learning framework that periodically retrains edge AI models to address deployment data drift.
- Jointly schedules inference and retraining tasks to maximize long-term inference accuracy under limited GPU resources.



□ Limitations

- Requires online DNN retraining, which incurs significant GPU and computation overhead.
- Optimizes model accuracy only rather than balancing multiple service-level objectives.

Evaluation Plan

❑ Set up

Edge Devices: Jetson Orin Nano, Jetson AGX Xavier, RTX 2080 Ti, RTX A5000

Learning Framework: Contextual Bandit / History-Aware Policy

Predictor Models: Accuracy, Latency, Energy

❑ Metric

Multi-SLO Satisfaction Rate

Calibration Effectiveness

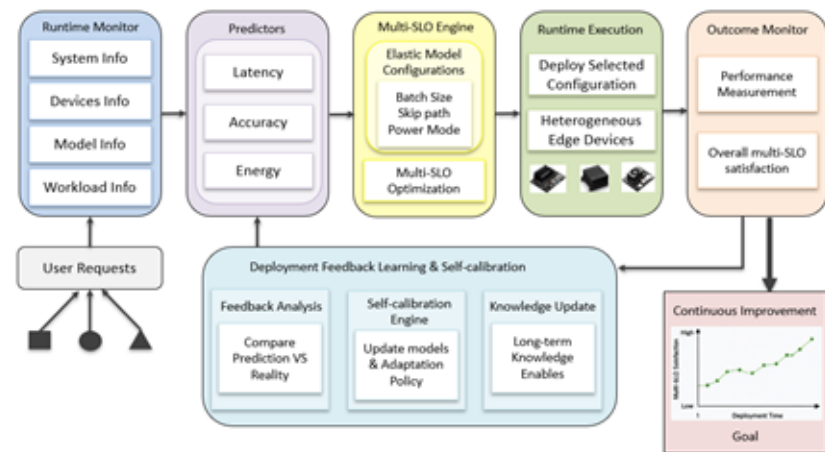
Cold Start

Runtime Overhead (CPU/Mem/Energy/Calibration Latency)

❑ Baselines

EdgeTimer

Ekya



Agenda

- Motivation and Research Challenges.

- Proposed Approach

1. Research Part #1 – Multi-SLO heterogeneous scheduling
2. Research Part #2 – Model-aware dynamic inference adaptation
3. Research Part #3 – Self-Calibrating Multi-SLO Adaptation
4. Evaluation Plan

- **Timeline, Summary, and Contributions**

Timeline

Semester:		1	2	3	4	5
Research	Thrust 1					
	Thrust 2					
	Thrust 3					
Paper	Paper Submission					
Thesis	Thesis Writing & Defense					

Table 2: Project Timeline

Summary and Contributions

□ Summary

- This project advances edge AI from static scheduling, to predictor-guided adaptation, and ultimately to self-calibrating deployment-feedback-driven intelligence for sustained multi-SLO satisfaction.

□ Research Contributions

- Develop a unified multi-SLO optimization framework
- Introduce predictor-guided model-aware adaptation
- Develop a deployment-feedback-driven self-calibration framework
- Design a lightweight drift-aware configuration re-ranking mechanism

Thank you!